

建图的时候要注意复杂度

注意重边的情况，如果是这样的话如果重边本身对算法有影响就先对数据预处理再建图

最大化最小值几乎一定考虑二分答案 T

可以用lru_cache来用dfs代替dp（当然还是能dp就dp啦）

算法部分

欧拉路径算法

欧拉路径就是哈密顿路问题

```
# n 顶点数, m 边数
# g是 {u:[v,i]} (i是边的id)
used = [False] * m
path = []
def dfs(u):
    while g[u]:
        v, e_id = g[u].pop()
        if used[e_id]:
            continue
        used[e_id] = True
        dfs(v)
    path.append(u)
dfs(start)
path.reverse()
```

背包问题

01背包

每个物品只能选一次。

```
dp = [0] * (m + 1)
for v, w in items:
    for j in range(m, v - 1, -1):
        dp[j] = max(dp[j], dp[j - v] + w)
```

完全背包

每个物品无限个。

```

dp = [0] * (m + 1)
for v, w in items:
    for j in range(v, m + 1):
        dp[j] = max(dp[j], dp[j - v] + w)

```

多重背包（朴素）

每种最多 s 个。

```

for v, w, s in items:
    for j in range(m, -1, -1):
        for k in range(1, s + 1):
            if j >= k * v:
                dp[j] = max(dp[j],
                           dp[j - k * v] + k * w)

```

多重背包（二进制优化）

```

new_items = []
for v, w, s in items:
    k = 1
    while k <= s:
        new_items.append((k * v, k * w))
        s -= k
        k <= 1
    if s:
        new_items.append((s * v, s * w))
# 然后当成 01 背包:
for v, w in new_items:
    for j in range(m, v - 1, -1):
        dp[j] = max(dp[j], dp[j - v] + w)

```

树形dp

没有相邻节点入选

```

def dfs(u, fa):
    # 选当前点，初始加上自身权值
    dp[u][1] = val[u]
    for v in g[u]:
        if v == fa:
            continue
        dfs(v, u)
    # dp[u][0]: 不选节点 u 时，以 u 为根的子树最大快乐值
    dp[u][1] += dp[v][0]
    # dp[u][1]: 选择节点 u 时，以 u 为根的子树最大快乐值
    dp[u][0] += max(dp[v][0], dp[v][1])
dfs(root, -1)

```

课程有前置课程，选的课程有限，最多能获得多少学分

```

# score 学分的list
# 1-index, 0作为超级源点
dp = [[0] * (m + 1) for _ in range(n + 1)]
sz = [0] * (n + 1)
def dfs(u):
    sz[u] = 1
    dp[u][1] = score[u]
    for v in g[u]:
        dfs(v)
        for j in range(sz[u], 0, -1):
            for k in range(1, sz[v] + 1):
                if j + k <= m:
                    dp[u][j + k] = max(
                        dp[u][j + k],
                        dp[u][j] + dp[v][k]
                    )
    sz[u] += sz[v]
dfs(0)

```

最近公共祖先(LCA)算法

```
class Solution:
    def lowestCommonAncestor(self, root: 'TreeNode', p: 'TreeNode', q: 'TreeNode') -> 'TreeNode':
        # 1. 如果 root 为空, 或者 root 就是我们要找的节点之一, 直接返回 root
        if not root or root == p or root == q:
            return root
        # 2. 递归在左子树和右子树中查找
        left = self.lowestCommonAncestor(root.left, p, q)
        right = self.lowestCommonAncestor(root.right, p, q)
        # 3. 如果左子树找到了一个, 右子树也找到了一个
        # 说明 p 和 q 分居 root 的两侧, root 就是 LCA
        if left and right:
            return root
        # 4. 如果只有左子树找到了, 返回左子树的结果
        if left:
            return left
        # 5. 如果只有右子树找到了, 返回右子树的结果
        if right:
            return right
        # 6. 如果都没找到, 返回 None
        return None
```

```
parent = [0] * (n + 1)
depth = [0] * (n + 1)
def dfs(u, fa):
    parent[u] = fa
    for v in g[u]:
        if v == fa:
            continue
        depth[v] = depth[u] + 1
        dfs(v, u)
dfs(root, 0)
# 求LCA (暴力版)
while depth[x] > depth[y]:
    x = parent[x]
while depth[y] > depth[x]:
    y = parent[y]
while x != y:
    x = parent[x]
    y = parent[y]
lca = x
```

关键路径算法

为防止出现一些问题，可思考建立一个超级根和超级叶

边活动（Activity On Edge, AOE）网

AOE网络中的最长路径被称为关键路径，把关键路径上的活动称为关键活动。

DAG，节点代表事件或里程碑，边代表活动，并且每条边有一个权重，表示完成该活动所需的时间。

step1:计算最早开始时间 (Earliest Start Time, EST)

使用拓扑排序遍历图

$$EST[v] = \max(EST[v], EST[u] + \text{weight}(u, v))$$

step2:计算最晚开始时间 (Latest Start Time, LST)

反向遍历拓扑排序后的图

$$LST[v] = \min(LST[v], LST[u] - \text{weight}(v, u))$$

```

from collections import defaultdict, deque
def critical_path(n, edges):
    """
    n: 节点数
    edges: 边列表
           [(u, v, w), ...]
    """
    # 建图
    graph = defaultdict(list)
    # 入度
    in_degree = [0] * n
    for u, v, w in edges:
        graph[u].append((v, w))
        in_degree[v] += 1
    # 拓扑排序 + 求 ve
    queue = deque()
    for i in range(n):
        if in_degree[i] == 0:
            queue.append(i)
    topo_order = []
    # 最早发生时间
    ve = [0] * n
    while queue:
        u = queue.popleft()
        topo_order.append(u)
        for v, w in graph[u]:
            # 更新 ve
            ve[v] = max(ve[v], ve[u] + w)
            # 入度减1
            in_degree[v] -= 1
            if in_degree[v] == 0:
                queue.append(v)
    # 逆拓扑求 vl
    # 初始化为终点最早时间
    vl = [max(ve)] * n
    for u in reversed(topo_order):
        for v, w in graph[u]:
            # 更新 vl
            vl[u] = min(vl[u], vl[v] - w)
    # 找关键活动
    critical_edges = []
    for u, v, w in edges:
        # 活动最早开始时间

```

```

e = ve[u]
# 活动最晚开始时间
l = vl[v] - w
if e == l:
    critical_edges.append((u, v, w))
return ve, vl, critical_edges

```

广度优先搜索 (bfs)

```

from collections import deque
def bfs(start):
    q = deque()
    q.append((start, 0))
    visited = {start}      # 必须用set, 否则会mle
    prev = {start: None}
    while q: # 强迫自己必须只能套一步while
        cur, step = q.popleft()
        if is_target(cur):
            path = []
            node = cur
            while node is not None:
                path.append(node)
                node = prev[node]
            path.reverse()
            print("最短路径:", path)
            return step
        for nxt in get_neighbors(cur):
            if nxt not in visited:
                visited.add(nxt)
                prev[nxt] = cur
                q.append((nxt, step+1))
    return -1 # 找不到

```

最小生成树算法 (MST: Minimum Spanning Tree)

- prim算法
用于找到连接所有顶点的最小生成树。

```

import heapq
def prim(n, adj):
    """
    adj: 邻接表, 格式为 {u: [(weight, v), ...]}
    """
    # mst_weight 存储最小生成树的总权重
    mst_weight = 0
    # visited 记录节点是否已经加入生成树
    visited = [False] * n
    # pq 是优先队列, 存储格式为 (weight, to_node)
    pq = [(0, 0)] # 从顶点 0 开始, 权重为 0
    nodes_count = 0
    while pq and nodes_count < n:
        weight, u = heapq.heappop(pq)
        # 如果点已经访问过, 跳过
        if visited[u]:
            continue
        # 将点标记为已访问, 并累加权重
        visited[u] = True
        mst_weight += weight
        nodes_count += 1
        # 遍历当前点的所有邻居
        for next_weight, v in adj[u]:
            if not visited[v]:
                heapq.heappush(pq, (next_weight, v))
    # 如果加入的点数等于 n, 说明生成了完整的树
    return mst_weight if nodes_count == n else -1

```

- Kruskal算法 / 并查集：用于找到连接所有顶点的最小生成树，适用于边集合已经给定的情况。


```

def find(i):
    if parent[i] != i:
        parent[i] = find(parent[i]) # 路径压缩
    return parent[i]
def union(i, j):
    root_i = find()
    root_j = find(j)
    if root_i != root_j:
        parent[root_i] = root_j
        return True
    return False
def kruskal(n, edges):
    # 按权重从小到大排序
    edges.sort()
    # 初始化 parent 数组 (代替类的 self.parent)
    parent = list(range(n))
    mst_weight = 0
    mst_edges = []
    for weight, u, v in edges:
        if union(u, v):
            mst_weight += weight
            mst_edges.append((u, v, weight))
            # 凑够 n-1 条边提前结束
            if len(mst_edges) == n - 1:
                break
    return mst_weight, mst_edges

```

01生成树问题

```

from collections import deque
n, m = map(int, input().split())
bad = [set() for _ in range(n + 1)]
for _ in range(m):
    u, v = map(int, input().split())
    bad[u].add(v)
    bad[v].add(u)
unvisited = set(range(1, n + 1))
cc = 0
while unvisited:
    start = unvisited.pop()
    cc += 1
    q = deque([start])
    while q:
        u = q.popleft()
        nxt = []
        for v in unvisited:
            if v not in bad[u]:
                nxt.append(v)
        for v in nxt:
            unvisited.remove(v)
            q.append(v)
print(cc - 1)

```

最短路径算法

- Dijkstra算法
带非负权边的图中，用于找到两个顶点之间的最短路径。

```

import heapq
def dijkstra(graph, start):
    dist = [float('inf')] * n
    dist[start] = 0
    pq = [(0, start)] # (当前距离, 节点)
    while pq:
        current_dist, node = heapq.heappop(pq)
        # 如果当前距离不是最优的, 跳过
        if current_dist > dist[node]:
            continue
        for neighbor, weight in graph[node]:
            new_dist = current_dist + weight
            if new_dist < dist[neighbor]:
                dist[neighbor] = new_dist
                heapq.heappush(pq, (new_dist, neighbor))
    return dist

```

用于同余最短路

```

def gcd(x, y):
    while y:
        x, y = y, x % y
    return x
# Scale by gcd so the gcd becomes 1 for the residue calculations.
scaled_coins = [v // g for v in coins]
smallest = scaled_coins[0]
INF = float("inf")
dist = [INF] * smallest
dist[0] = 0
heap = [(0, 0)]
edges = [v for v in scaled_coins if v != smallest]
while heap:
    d, r = heapq.heappop(heap)
    if d != dist[r]:
        continue
    for w in edges:
        nr = (r + w) % smallest
        nd = d + w
        if nd < dist[nr]:
            dist[nr] = nd
            heapq.heappush(heap, (nd, nr))

```

- Bellman-Ford算法

用于处理带有负权边的图的最短路径问题。

```
def bellman_ford(graph, V, src):  
    # 初始化从源点到所有其他顶点的距离为无穷大  
    dist = [float("inf")] * V  
    dist[src] = 0  
    # 对所有边进行 V - 1 次松弛操作（路径经过的边数最多为 V - 1 条）  
    for _ in range(V - 1):  
        for u, v, w in graph:  
            if dist[u] != float("inf") and dist[u] + w < dist[v]:  
                dist[v] = dist[u] + w  
    # 检查负权环（如果再进行一次松弛还能更新，说明存在负权环）  
    for u, v, w in graph:  
        if dist[u] != float("inf") and dist[u] + w < dist[v]:  
            print("图中存在负权回路! ")  
            return None  
    return dist
```

spfa优化（Shortest Path Faster Algorithm）

```

from collections import deque
def bellman_ford_spfa(graph, V, src):
    # 初始化距离
    dist = [float('inf')] * V
    dist[src] = 0
    # 队列
    queue = deque([src])
    # 是否在队列中
    in_queue = [False] * V
    in_queue[src] = True
    # 入队次数（用于判负环）
    count = [0] * V
    count[src] = 1
    while queue:
        # 取出队首元素
        u = queue.popleft()
        in_queue[u] = False
        # 松弛 u 的所有邻边
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                # 如果 v 不在队列中
                if not in_queue[v]:
                    count[v] += 1
                    # 出现负环
                    if count[v] >= V:
                        print("图中存在负权回路！")
                        return None
                # SLF 优化
                if queue and dist[v] < dist[queue[0]]:
                    queue.appendleft(v)
                else:
                    queue.append(v)
            in_queue[v] = True
    return dist

```

处理差分约束

$x_{c_1} \leq x_{c_2} + y$ 直接建 $c_2 \rightarrow c_1$ 权值为 y 即可

```

from collections import deque
dist = [0] * (n + 1)      # 超级源点到所有点距离为0
cnt = [0] * (n + 1)       # 入队次数
inq = [True] * (n + 1)    # inqueue
q = deque(range(1, n + 1)) # 相当于跑完超级源点0
while q:
    u = q.popleft()
    inq[u] = False
    for v, w in g[u]:
        if dist[v] > dist[u] + w:
            dist[v] = dist[u] + w
            if not inq[v]:
                q.append(v)
                inq[v] = True
                cnt[v] += 1
                if cnt[v] > n:
                    print("NO")
                    sys.exit()
print(*dist[1:]) # 一个合理的解

```

处理差分约束，希望解的和最小，且每个解都 ≥ 1

$x_{c_1} \geq x_{c_2} + y$ 直接建 $c_2 \rightarrow c_1$ 权值为 y 即可，最长路

```

# 超级源点 0
for i in range(1, n + 1):
    g[0].append((i, 1))

```

- Floyd-Warshall算法：用于找到图中所有顶点之间的最短路径。

```

def floyd_warshall(graph):
    # graph 是一个邻接矩阵, 如果两点不连通, 值为 float('inf')
    # graph[i][i] = 0
    # 节点数量
    n = len(graph)
    # 初始化距离矩阵 dist
    dist = [list(row) for row in graph]
    # 核心算法: 三层循环
    # 注意: k (中间点) 必须在最外层!
    for k in range(n):
        for i in range(n):
            for j in range(n):
                # 如果通过中间点 k 的路径更短, 则更新
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist

```

如果要找从i到j的最短路径中i的下一个节点, 用next_node[i][j]维护 = next_node[i][k]

归并排序算法 (merge sort)

```

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)
def merge(left, right):
    res = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            res.append(left[i]); i += 1
        else:
            res.append(right[j]); j += 1
    res.extend(left[i:])
    res.extend(right[j:])
    return res

```

调度场算法 (shunting yard)

数字 扔进输出。(注意数字的读取)

左括号 入栈。

操作符 把栈里比自己同等级或强的都弹出去，然后自己入栈。

右括号 栈内“大清扫”直到遇见左括号。

最后 把栈里剩下的所有东西全部倒出来。

深度优先搜索算法 (dfs)

用于判无向图环

```
def has_cycle_dfs(graph, n):
    visited = [False] * n
    def dfs(i, parent): # 要记录每个节点的parent，因为这是无向图
        visited[i] = True
        for neighbor in graph[i]:
            if not visited[neighbor]:
                if dfs(neighbor, i):
                    return True
            elif neighbor != parent:
                return True
        return False
    for i in range(n):
        if not visited[i]:
            if dfs(i, -1):
                return True
    return False
```

拓扑排序算法 (Topological Sorting)

- Karn算法 / BFS :

用于对有向无环图进行拓扑排序

不断地移除图中的入度为0的顶点，并将其添加到拓扑排序的结果中，直到图中所有的顶点都被移除


```

from collections import deque, defaultdict
def karn(graph,n): # n是顶点数
    indegree = [0] * n
    result = []
    queue = deque()
    # 计算每个顶点的入度
    for u in graph:
        for v in graph[u]:
            indegree[v] += 1
    # 将入度为 0 的顶点加入队列
    for u in range(n):
        if indegree[u] == 0:
            queue.append(u)
    # 执行拓扑排序
    while queue:
        u = queue.popleft()
        result.append(u)
        for v in graph[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                queue.append(v)
    return result

```

用于dag判环

```

from collections import deque, defaultdict
def has_cycle_karn(graph,n): # n是顶点数
    indegree = [0] * n
    num_visited = 0
    queue = deque()
    # 计算每个顶点的入度
    for u in graph:
        for v in graph[u]:
            indegree[v] += 1
    # 将入度为 0 的顶点加入队列
    for u in range(n):
        if indegree[u] == 0:
            queue.append(u)
    # 执行拓扑排序
    while queue:
        u = queue.popleft()
        num_visited += 1
        for v in graph[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                queue.append(v)
    return num_visited != n

```

拓扑排序要求序号从小到大时把deque换成heapq

- DFS:
用于对有向无环图（DAG）进行拓扑排序

```

def topo_dfs(graph,n):
    visited = [False] * n
    result = []
    def dfs(u):
        if visited[u]:
            return
        visited.add(u)
        # 递归访问所有邻居
        for v in graph[u]:
            dfs(v)
        # 【关键点】在回溯阶段加入结果
        result.append(u)
    # 遍历图中所有节点（应对不连通的情况）
    for i in range(n):
        dfs(i)
    # 返回反转后的序列
    return result[::-1]

```

用于dag判环

用一个额外的递归栈记录当前路径上的节点，如果在当前DFS过程中再次访问到了路径上的某个节点，就说明存在环

这里的rec_stack可能在遍历完一个分支后还有一些元素未完全删除

```

def has_cycle_dfs(graph,n):
    visited = [False] * n
    rec_stack = set()
    def dfs(i):
        visited[i] = True
        rec_stack.add(i)
        for neighbor in graph[i]:
            if not visited[neighbor]:
                if dfs(neighbor):
                    return True
            elif neighbor in rec_stack:
                return True
        rec_stack.remove(i)
        return False
    for i in range(n):
        if not visited[i]:
            if dfs(i):
                return True
    return False

```

三色标记法

```

def has_cycle_dfs(graph,n):
    status = [0] * n # 0表示没访问, 1表示访问中, 2表示访问过了
    def dfs(i):
        status[i] = 1
        for neighbor in graph[i]:
            if status[neighbor] == 0:
                if dfs(neighbor):
                    return True
            elif status[neighbor] == 1:
                return True
        status[i] = 2
        return False
    for i in range(n):
        if status[i] != 2:
            if dfs(i):
                return True
    return False

```

强连通分量算法 (SCC)

强连通分量 (SCC: Strongly Connected Components) 是指有向图中的一个极大子图，其中任意两个节点都是相互可达的。

SCC好处在于可以缩点然后变成有向无环图 (DAG) 进行拓扑排序

凡是研究各个点能到哪些点的题一定要想到SCC

注意特判：比如只有一个强连通块，比如只有一块入度为0，比如只有一块出度为0

- Kosaraju算法 / 2 DFS:

第一次DFS：标准的深度优先搜索，记录下顶点完成搜索的顺序，找出每个顶点的完成时间

第二次DFS：在反向图中我们按照第一步中记录的顶点完成时间的逆序进行DFS，找出所有强连通分量。

```

def kosaraju(n, adj):
    # 1. 正向 DFS, 记录完成顺序
    visited = [False] * n
    stack = []
    def dfs1(u):
        visited[u] = True
        for v in adj[u]:
            if not visited[v]:
                dfs1(v)
        stack.append(u) # 回溯时压入栈
    for i in range(n):
        if not visited[i]:
            dfs1(i)
    # 2. 创建反向图
    rev_adj = [[] for _ in range(n)]
    for u in range(n):
        for v in adj[u]:
            rev_adj[v].append(u)
    # 3. 反向 DFS, 提取 SCC
    visited = [False] * n
    sccs = []
    def dfs2(u, current_scc):
        visited[u] = True
        current_scc.append(u)
        for v in rev_adj[u]:
            if not visited[v]:
                dfs2(v, current_scc)
    while stack:
        u = stack.pop()
        if not visited[u]:
            current_scc = []
            dfs2(u, current_scc)
            sccs.append(current_scc)
    return sccs

```

- Tarjan算法（只需要一次 DFS，不需要反转图）：

搜索次序表示顶点被首次访问的次序，最低链接值表示从当前顶点出发经过一系列边能到达的搜索次序最早的顶点的搜索次序。

输出的本身就是拓扑排序的逆

```

def tarjan_scc(n, adj):
    dfn = [-1] * n      # 搜索次序
    low = [-1] * n      # 最低链接值
    stack = []          # 辅助栈
    in_stack = [False] * n
    timer = 0
    sccs = []          # 存储最终结果
    def dfs(u):
        nonlocal timer
        dfn[u] = low[u] = timer
        timer += 1
        stack.append(u)
        in_stack[u] = True
        for v in adj[u]:
            if dfn[v] == -1: # 情况 A: 邻居未访问
                dfs(v)
                low[u] = min(low[u], low[v])
            elif in_stack[v]: # 情况 B: 邻居在栈中 (回边)
                low[u] = min(low[u], dfn[v])
        # 判定强连通分量的根
        if low[u] == dfn[u]:
            current_scc = []
            while True:
                node = stack.pop()
                in_stack[node] = False
                current_scc.append(node)
                if node == u:
                    break
            sccs.append(current_scc)
    for i in range(n):
        if dfn[i] == -1:
            dfs(i)
    return sccs

```

两种方法之后缩点变成dag，下面给出代码

```

# 给每个点分配SCC编号
scc_id = [0] * n
for i in range(len(sccs)):
    for node in sccs[i]:
        scc_id[node] = i
# 构建缩点后的 DAG & 计算出度
dag = [[] for _ in range(len(sccs))]
out_degree = [0] * len(sccs)
edges = set() # 防止重复建边
for u in range(n):
    for v in adj[u]:
        a = scc_id[u]
        b = scc_id[v]
        if a != b and (a, b) not in edges:
            edges.add((a, b))
            dag[a].append(b)
            out_degree[a] += 1
# sccs: 所有强连通分量;scc_id: 每个点属于哪个scc;dag: 缩点后的有向无环图;out_degree: 每个缩点的出度

```

scc从一个指定点到另一个指定点


```

cnt = len(sccs)
start = scc_id[0]
end = scc_id[n - 1]
# start 可达
reach1 = [False] * cnt
def dfs_start(u):
    reach1[u] = True
    for v in dag[u]:
        if not reach1[v]:
            dfs_start(v)
dfs_start(start)
# 能到 end
reach2 = [False] * cnt
def dfs_end(u):
    reach2[u] = True
    for v in rdag[u]:
        if not reach2[v]:
            dfs_end(v)
dfs_end(end)
# 拓扑排序
q = deque()
deg = indegree[:]
for i in range(cnt):
    if deg[i] == 0:
        q.append(i)
topo = []
while q:
    u = q.popleft()
    topo.append(u)
    for v in dag[u]:
        deg[v] -= 1
        if deg[v] == 0:
            q.append(v)
# DP
INF = 10**18
mn = [INF] * cnt # 维护到一个地方途径的所有地方的一个性质
mn[start] = scc_min[start]
for u in topo:
    if mn[u] == INF:
        continue
    if not (reach1[u] and reach2[u]):
        continue
    for v in dag[u]:

```

```

        if not (reach1[v] and reach2[v]):
            continue
        mn[v] = min(
            mn[v],
            min(mn[u], scc_min[v])
        )
    ans = 0
    for i in range(cnt):
        if reach1[i] and reach2[i] and mn[i] != INF:
            ans = max(
                ans,
                scc_max[i] - mn[i]
            )

```

Morris 遍历 (Morris Traversal) 算法

$O(1)$ 额外空间、 $O(n)$ 时间

对每个节点 cur:

若 cur 无左子树: 直接访问, 向右走。

若 cur 有左子树: 找到 左子树最右节点 mostRight (中序前驱)。

若 mostRight.right == null: 第一次到 cur → 让 mostRight.right = cur (建线索), cur 向左走。

若 mostRight.right == cur: 第二次到 cur → 恢复 mostRight.right = null (拆线索), cur 向右走。

```
def morris_inorder(root):
    res = []
    cur = root
    while cur:
        # 情况1: 没有左子树, 直接访问, 然后往右走
        if not cur.left:
            res.append(cur.val)
            cur = cur.right
        # 情况2: 有左子树, 找到左子树最右边节点 (前驱)
        else:
            prev = cur.left
            while prev.right and prev.right != cur:
                prev = prev.right
            # 第一次来到 cur: 建立线索, 让 prev.right = cur
            if not prev.right:
                prev.right = cur
                cur = cur.left
            # 第二次来到 cur: 说明左子树遍历完了, 访问 cur, 恢复树
            else:
                prev.right = None
                res.append(cur.val)
                cur = cur.right
    return res
```

```

def morris_preorder(root):
    res = []
    cur = root
    while cur:
        if not cur.left:
            res.append(cur.val)
            cur = cur.right
        else:
            prev = cur.left
            while prev.right and prev.right != cur:
                prev = prev.right
            if not prev.right:
                prev.right = cur
                res.append(cur.val) # 这里提前访问
                cur = cur.left
            else:
                prev.right = None
                cur = cur.right
    return res

```

欧拉筛

保证每个合数只被它最小的质因数标记一次。

还可以用于既取出所有质数还监测所有数的最小质因数，可以用来因式分解

```

n = int(input())
is_prime = [True] * (n+1)
primes = []
for i in range(2, n+1):
    if is_prime[i]:
        primes.append(i)
    for p in primes:
        if i * p > n:
            break
        is_prime[i * p] = False
        if i % p == 0: #这一步保证p一定是标记的合数的最小质因子
            break

```

埃氏筛

把 p 的倍数标记为非质数。

```

n = 30
is_prime = [True] * (n+1)
is_prime[0] = is_prime[1] = False
for i in range(2, int(n**0.5)+1):
    if is_prime[i]:
        for j in range(i*i, n+1, i): # 注意从 i*i 开始
            is_prime[j] = False
primes = [i for i in range(2, n+1) if is_prime[i]]

```

数据结构部分

单调栈 (Monotonic Stack)

寻找下一个更大的元素(用栈存储index, 比栈顶对应的数小的保留, 大的就逐项更新比较pop)

寻找下一个更小的元素

直方图中的最大矩形(思路本质就是在处理每个柱子作为右边界的情形)

```

def checkmaxsize(mylist):
    stack = [-1] # 防止最后栈里第一个元素无法追溯其可以开始的地方
    mymax = 0
    for index,num in enumerate(mylist):
        while stack[-1] != -1 and mylist[stack[-1]] >= num:
            # 这里必须用>=, 因为剩下的必须严格单调递增
            a = stack.pop() # 否则无法追溯左端点
            mymax = max(mymax, mylist[a]*(index-stack[-1]-1))
            # pop掉的数在[stack[-1]+1, index-1]是最小值
        stack.append(index)
    while stack[-1] != -1:
        a = stack.pop()
        mymax = max((len(mylist)-1-stack[-1])*mylist[a], mymax)
        # pop掉的数在[stack[-1]+1, len(mylist)-1]是最小值
    return mymax

```

维护一个区间, 左端最小, 右端最大

```

from bisect import bisect_right
n = int(data[0])
# h是题目给的list
max_stk = [] # 对j找左边离它最近的且价格 ≥ h[j]的位置在哪里
buy_stk = [] # 维护所有可能的买入点
for j in range(n):
    cur = h[j]
    # 找左边最近的 >= cur 的位置
    while max_stk and h[max_stk[-1]] < cur:
        max_stk.pop()
    left_barrier = max_stk[-1] if max_stk else -1
    # 维护买入点候选集
    while buy_stk and h[buy_stk[-1]] >= cur:
        buy_stk.pop()
    # 找最靠左的合法买入点
    if buy_stk:
        idx = bisect_right(buy_stk, left_barrier)
        if idx < len(buy_stk):
            best_i = buy_stk[idx]
    max_stk.append(j)
    buy_stk.append(j)

```

哈夫曼树 (Huffman Tree)

可以正常建树

也可以考虑用heapq直接做

前缀树 (trie)

```
class Trie:
    def __init__(self):
        self.tree = {}

    def insert(self, word: str) -> None:
        cur = self.tree
        for c in word:
            if c not in cur:
                cur[c] = {}
            cur = cur[c]
        # 单词结束标记 (用 None 表示结尾)
        cur[None] = True

    def search(self, word: str) -> bool:
        cur = self.tree
        for c in word:
            if c not in cur:
                return False
            cur = cur[c]
        # 判断是否是完整单词
        return (None in cur)

    def startsWith(self, prefix: str) -> bool:
        cur = self.tree
        for c in prefix:
            if c not in cur:
                return False
            cur = cur[c]
        return True
```

线段树 (segment tree)

处理任何区间类问题

将一个大区间 $O(n)$ 的查询, 拆解成若干个已经预处理好的小区间 $O(\log n)$ 的拼接。

对[1,2,3, 4]建树

[1,4],

[1,2],[3,4]

[1,1],[2,2],[3,3],[4,4]

#先将序列长度扩为2的幂次再进行操作，也就是让n为2的幂

```
n = len(arr)
size = 1
while size < n:
    size <<= 1
n = size
tree = [0] * (2*n)
def build(arr, n):
    for i in range(n):
        tree[n+i] = arr[i]
    for i in range(n-1, 0, -1):
        # 2*i 是左孩子, 2*i + 1 是右孩子
        tree[i] = tree[2*i] + tree[2*i+1]
def updateTreeNode(p, value, n):
    p = p+n
    tree[p] = value
    i = p
    while i > 1:
        i = i//2
        # 父节点等于左右两个子节点之和
        tree[i] = tree[2*i] + tree[2*i+1]
def query(l, r, n): #[l, r)
    res = 0
    l += n
    r += n
    while l < r:
        if (l % 2 != 0):
            res += tree[l]
            l += 1
        if (r % 2 != 0):
            r -= 1
            res += tree[r] #这里的r是已经被减过了的
        l = l // 2
        r = r // 2
    return res
```

并查集 (Disjoint Set)

用于无向图判环

Find (path compression) 每次操作后都把沿途的每个节点直接挂到根上


```

Parent = [i for i in range(n)]
def find(i):
    if (Parent[i] == i):
        return i
    else:
        result = find(Parent[i])
        Parent[i] = result
        return result

```

union可以被rank或size优化

```

rank = [1]*n
def union(i,j):
    i1 = find(i)
    j1 = find(j)
    if i1 == j1:
        return
    if rank[i1] < rank[j1]:
        Parent[i1] = j1
    elif rank[i1] > rank[j1]:
        Parent[j1] = i1
    else:
        Parent[j1] = i1
        rank[i1] += 1

```

```

size = [1]*n
def union(i, j):
    i1 = find(i)
    j1 = find(j)
    if i1 == j1:
        return
    if size[i1] < size[j1]:
        Parent[i1] = j1
        size[j1] += size[i1]
    else:
        Parent[j1] = i1
        size[i1] += size[j1]

```

用于k分图

下面代码认为i, j结合意味着 (j的种类-i的种类) 模k余1

```

Parent = [i for i in range(n)]
type = [0]*n
def find(i):
    if (Parent[i] == i):
        return i
    else:
        result = find(Parent[i])
        type[i] = (type[i]+type[Parent[i]])%k #这里是找以前的父节点，后面率先更新的type应该是以
        Parent[i] = result
        return result
def union(i,j):
    x = find(i)
    y = find(j)
    if x == y:
        return type[j] - type[i] == 1
    Parent[y] = x
    type[y] = (1+type[i]-type[j])%k
    return True

```

维护members的同时节约空间

```

rx = find(x)
ry = find(y)
if rx != ry:
    if len(members[rx]) < len(members[ry]):
        rx, ry = ry, rx
    parent[ry] = rx
    members[rx].extend(members[ry])
    members[ry].clear()
    size[rx] += size[ry]
    pile_count -= 1
    if size[rx] >= s:
        collapsed = members[rx]
        pile_count += size[rx] - 1
        for block in collapsed:
            parent[block] = block
            size[block] = 1
            members[block] = [block]
        collapsed.clear()

```

栈 (stack)

用栈进行中序遍历

```
class Solution:
    def inorderTraversal(self, root: Optional[TreeNode]) -> List[int]:
        res = []
        stack = []
        curr = root
        while curr or stack:
            # 1. 尽可能向左走，并将沿途节点入栈
            while curr:
                stack.append(curr)
                curr = curr.left
            # 2. 当前节点为空，说明左边走到底了，弹出栈顶元素（最近的根节点）
            curr = stack.pop()
            res.append(curr.val)
            # 3. 转向右子树
            curr = curr.right
        return res
```

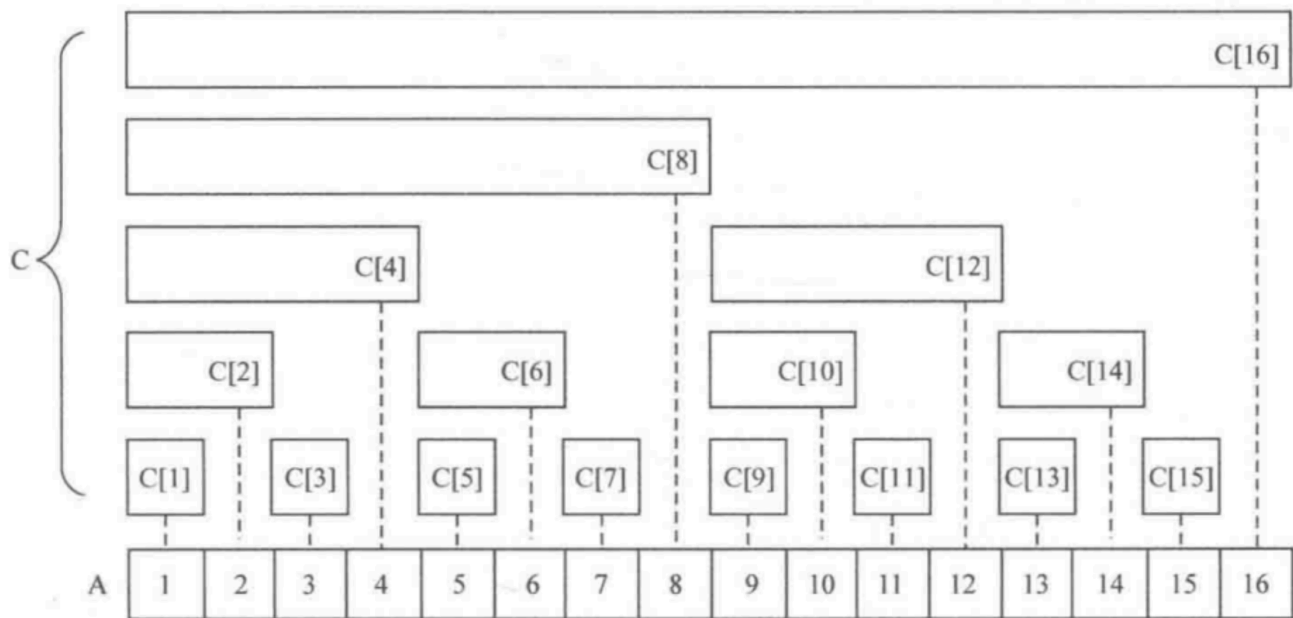
用递归模拟进出栈

```
n = int(input())
def dfs(push_cnt, stack_cnt, pop_cnt):
    # 已经全部出栈
    if pop_cnt == n:
        return
    # 还能进栈
    if push_cnt < n:
        dfs(push_cnt + 1, stack_cnt + 1, pop_cnt)
    # 栈非空，可以出栈
    if stack_cnt > 0:
        dfs(push_cnt, stack_cnt - 1, pop_cnt + 1)
dfs(0, 0, 0)
```

树状数组 (BIT/Fenwick Tree)

单点修改和前缀和查询

动态前缀和、逆序对计数、区间更新+单点查询（配合差分）



Binary Indexed Tree

```
class BIT:
```

```
    def __init__(self, n):
```

```
        self.size = n
```

```
        self.tree = [0] * (n + 1)
```

```
    def lowbit(self, x):
```

```
        return x & -x
```

```
    def update(self, i, delta):
```

```
        """将第 i 个元素增加 delta"""
```

```
        while i <= self.size:
```

```
            self.tree[i] += delta
```

```
            i += self.lowbit(i)
```

```
    def query(self, i):
```

```
        """查询前 i 个元素的和"""
```

```
        s = 0
```

```
        while i > 0:
```

```
            s += self.tree[i]
```

```
            i -= self.lowbit(i)
```

```
        return s
```

链表 (Linked List)

记住可以用dummy作为哨兵节点

链表反转

```

def reverse_linked_list(head: ListNode) -> ListNode:
    prev = None
    curr = head
    while curr is not None:
        prev = ListNode(curr.val, prev)
        curr = curr.next
    return prev

def reverse_linked_list(head: ListNode) -> ListNode:
    prev = None
    curr = head
    while curr is not None:
        next_node = curr.next # 暂存当前节点的下一个节点
        curr.next = prev      # 将当前节点的下一个节点指向前一个节点
        prev = curr           # 前一个节点变为当前节点
        curr = next_node      # 当前节点变更为原先的下一个节点
    return prev

```

合并两个排序链表

```

def merge_sorted_lists(l1, l2):
    dummy = Node(0) #dummy (哑节点 / 哨兵节点) 能避免处理链表头节点为空的边界情况
    tail = dummy
    while l1 and l2:
        if l1.data < l2.data:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next
    if l1:
        tail.next = l1
    else:
        tail.next = l2
    return dummy.next

```

查找链表的中间节点 (快慢指针)

```
def find_middle_node(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

1->1;2->2;3->2;4->3;5->3;n->(n//2)+1

双向链表

```
class Node:
    def __init__(self, data):
        self.data = data # 节点数据
        self.next = None # 指向下一个节点
        self.prev = None # 指向前一个节点
```

循环链表可以用快慢指针去找圈

测试链表

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next
def construct(mylist):
    head = ListNode()
    cur = head
    for num in mylist:
        cur.next = ListNode(num)
        cur = cur.next
    return head.next
def show(head):
    result = []
    while head:
        result.append(head.val)
        head = head.next
    return result
```

语法部分

位运算

按位与 AND (&) (两个位都为 1 → 结果为 1, 否则为 0)
按位或 OR (|) (只要有一个位为 1 → 结果为 1, 两个都是 0 才为 0)
按位异或 XOR (^) (两位相同 → 0, 不同 → 1)
按位取反 NOT (~)
~n 在二进制表示是把n的0改成1, 1改成0
-n = ~n + 1

n: 是一个整数, 比如 n = 12, 二进制是 1100
-n: 在计算机中用补码表示, 就是 ~n + 1
n = 00001100
~n = 11110011
-n = 11110100 (相当于取反+1)

- 检查i是否为2的幂
if i & (i - 1) == 0:
- 提取一个整数在二进制表示下, 最低位的那个 1 所代表的数值。
 $lowbit(x) = x \& (-x)$

常见 is 开头的方法及作用

方法名	功能描述	示例	结果
isupper()	判断字符串是否全部为大写字母	"ABC".isupper()	True
islower()	判断字符串是否全部为小写字母	"abc".islower()	True
isdigit()	判断字符串是否全部由数字组成	"123".isdigit()	True
isalpha()	判断字符串是否全部由字母组成	"abc".isalpha()	True
isalnum()	判断字符串是否由字母和数字组成	"abc123".isalnum()	True
isspace()	判断字符串是否全部由空白字符组成	" \t\n".isspace()	True
istitle()	判断字符串是否为标题格式 (首字母大写)	"Hello World".istitle()	True

" ".islower() 的结果是 False

```
print("123 !".upper())      # 输出: 123 !
print(" ".upper())          # 输出:   (还是空格)
```

upper() 只转换字母，不改变数字、空格和符号。

math

里面主要有：数学常量、基础函数、三角函数、指数对数函数、取整函数、特殊函数等。

```
math.pi      # 圆周率  $\pi = 3.14159...$ 
math.e        # 自然常数  $e = 2.71828...$ 
```

```
math.pow(x, y)    # 幂运算  $x^y$ 
math.factorial(n) # 阶乘  $n!$ 
math.gcd(a, b)    # 最大公约数
```

```
math.ceil(x)      # 向上取整
```

```
math.exp(x)       #  $e^x$ 
math.log(x)        #  $\ln(x)$ , 自然对数
math.log10(x)      # 以 10 为底的对数
math.log2(x)       # 以 2 为底的对数
```

```
math.sin(x)       # 正弦,  $x$  是弧度
math.cos(x)       # 余弦
math.tan(x)       # 正切
math.asin(x)      # 反正弦 (返回弧度)
math.acos(x)      # 反余弦
math.atan(x)      # 反正切
math.atan2(y, x)  # 用坐标  $(y, x)$  求角度
```

角度与弧度转换：

```
math.radians(deg) # 角度  $\rightarrow$  弧度
math.degrees(rad) # 弧度  $\rightarrow$  角度
```



```
math.sinh(x)    # 双曲正弦
math.cosh(x)    # 双曲余弦
math.tanh(x)    # 双曲正切
```

```
math.comb(n, k) # 组合数 C(n,k)
math.perm(n, k) # 排列数 P(n,k)
```

bin(x)

把整数 x 转换为二进制字符串。

格式固定为：

'0bxxxxx'

其中 0b 是前缀，表示“二进制”。

from itertools import permutations

生成所有排列

```
text = "ABC"
result = list(permutations(text))
print(result)
# 输出: [('A', 'B', 'C'), ('A', 'C', 'B'), ('B', 'A', 'C'),
#        ('B', 'C', 'A'), ('C', 'A', 'B'), ('C', 'B', 'A')]
```

ord和chr

ord("A") = 65

ord("Z") = 90

ord("a") = 97

ord("z") = 122

chr(65) = "A"

s.replace(old, new, count)

old：要被替换掉的子串

new：新的子串

count（可选）：替换的最大次数，省略时表示替换全部

```
s = "hello world world"
# 替换所有 "world" 为 "python"
print(s.replace("world", "python"))
# 输出: hello python python
# 只替换前 1 个
print(s.replace("world", "python", 1))
# 输出: hello python world
# 原字符串不变
print(s)
# 输出: hello world world
```

格式说明符详解

- 对齐方式与宽度

说明符	含义	示例	输出
<	左对齐	f"{'hi':<6}"	'hi '
>	右对齐（默认）	f"{'hi':>6}"	' hi'
^	居中对齐	f"{'hi':^6}"	' hi '
数字	总宽度（字符数）	f"{123:6}"	' 123'
=	填充符后数字前	f"{42:=+5}"	'+ 42'

- 数字格式化

类型	含义	示例	输出
d	十进制整数	f"{123:d}"	123
f	浮点数（默认6位小数）	f"{3.14:f}"	3.140000
.nf	保留 n 位小数	f"{3.14:.2f}"	3.14
%	百分比（自动乘 100）	f"{0.85:%}"	85.000000%
.n%	百分比保留 n 位小数	f"{0.85:.1%}"	85.0%
e	科学计数法（小写 e）	f"{12345:e}"	1.234500e+04
E	科学计数法（大写 E）	f"{12345:E}"	1.234500E+04

类型	含义	示例	输出
g	自动切换普通/科学计数法	f"{12345:g}"	12345

- 千分位分隔符

```
print(f"{1234567:,}")          # 输出: 1,234,567
print(f"{1234567.89:,.2f}")    # 输出: 1,234,567.89
```

- 填充字符
默认填充格，你也可以自定义填充字符：

```
print(f"{'hi':*^10}")          # 输出: ***hi***
print(f"{42:0>5}")             # 输出: 00042
```

- 符号控制（整数/浮点）

符号	含义	示例	输出
+	总是显示正负号	f"{42:+}"	+42
-	仅负数显示负号	f"{42:-}"	42
空格	正数前留空，负数显示负号	f"{42: }"	42

- 字符串处理格式

```
text = "Python"
print(f"{text:.3}")          # 输出: Pyt（截取前3个字符）
print(f"{text:>10}")          # 宽度10，右对齐
print(f"{text:*^10}")        # 居中，用*填充: **Python**
```

- 多项组合（格式说明可以组合使用）

```
x = 3.14159
print(f"{x:>10.2f}")          # 宽度10，右对齐，保留两位小数
# 输出:          3.14
```

- 进制格式化

格式	含义	示例	输出
b	二进制	f"{10:b}"	1010
o	八进制	f"{10:o}"	12
x	十六进制（小写）	f"{255:x}"	ff
X	十六进制（大写）	f"{255:X}"	FF
#	显示前缀	f"{255:#x}"	0xff

查找dict的第k个key

可以用 `itertools.islice` 按需取（更省内存）

```
from itertools import islice
d = {"a": 1, "b": 2, "c": 3}
k = 1
key = next(islice(d.keys(), k, k+1))
print(key) # b
```

字符串大小写互换

用 `string.swapcase()` 来实现

`dict.setdefault(key, default)`

这个是 `defaultdict` 用不了的选择

功能：

如果字典中存在 `key`，返回该 `key` 对应的 `value`。

如果字典中不存在 `key`，则将 `key` 添加到字典，并将其 `value` 设置为 `default`，然后返回 `default`。

```
d = {}
d.setdefault('x', []).append(10)
print(d) # {'x': [10]}
```